

SE 410 - Software Framework

Applications Project Report

Inventory and Order Management System for a Small Business

Assist. Prof. Dr. Serhat UZUNBAYIR

Authors

Erkan Efe Yeşilçimen - 20220601074

Ege Sezak - 20220601059

Mehmet Bora Böcekoğlu - 20220601010

Hasan Basri Karşlıoğlu - 20210601033

Inventory and Order Management System for a Small Business.....	0
1. Introduction.....	1
2. Project Objectives.....	1
3. Project Scope:.....	1
4. Requirements.....	1
4.1 Functional Requirements.....	1
4.2 Non-Functional Requirements.....	2
5. Database Schema.....	2
6. SQL Scripts.....	3
7. LINQ Queries.....	3
8. Delegates and Events.....	4
9. MVC Structure.....	5
9.1 Model.....	5
9.2 View.....	5
9.3 Controller.....	6
10. Sources.....	6
11. User Interface Design.....	6
12. Tests.....	10
13. Conclusion.....	11

1. Introduction

The Inventory and Order Management System is a comprehensive solution designed to help small businesses manage their inventory, track orders, and automate restocking processes. This system provides two distinct interfaces: a desktop application for staff members to manage inventory and process orders, and a web application for customers to browse products and place orders.

2. Project Objectives

1. Create a real-time inventory management system that prevents over-selling through atomic stock deduction.
2. Implement a role-based access control system with two user types: Staff (for administrative functions) and Customers (for placing orders).
3. Develop a robust REST API backend using ASP.NET Core with JWT-based authentication.
4. Enable comprehensive order tracking with multiple status stages (Pending, Processing, Shipped, Delivered, Cancelled).
5. Implement advanced reporting capabilities to provide insights into sales trends and top-selling products.
6. Provide staff with tools to manage inventory levels, update product information, and generate business reports.
7. Deliver a user-friendly desktop application for staff operations.

3. Project Scope:

- Version 1.0 focuses on the backend REST API and desktop application
- Version 2.0 will include the web application for customers
- Comprehensive unit testing using xUnit framework
- SQLite database for data persistence

4. Requirements

4.1 Functional Requirements

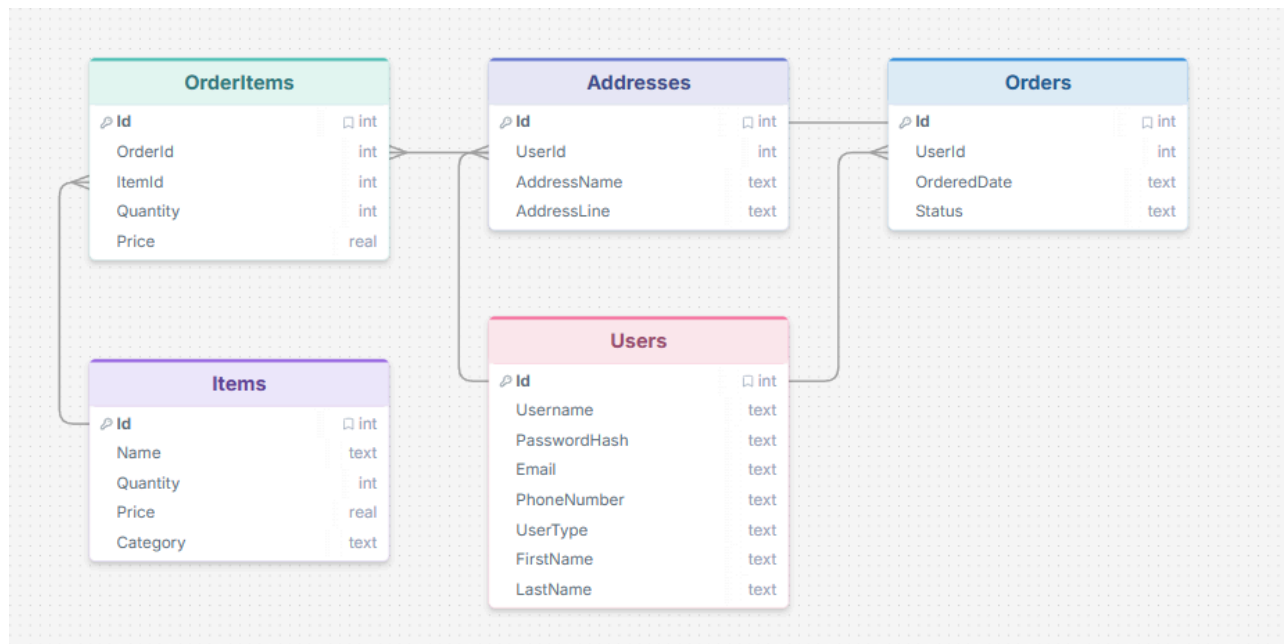
- 1- The system should allow users to search for items by name using partial string matching.
- 2- The system should allow users to sort search results by price and filter by category.
- 3- The system should display real-time availability status of an item to all users.
- 4- The system should restrict data modification and order placement to authenticated users only.
- 5- The system should allow customers to create accounts, log in, and manage their profiles.
- 6- The system should allow logged-in customers to place orders containing one or multiple items.
- 7- Customers should be able to view their active order status and a complete historical record of past orders.

- 8- The system should provide customers with a tracking page to view the current shipping stage of their orders.
- 9- The staff should be able to add, remove, and update item details and prices.
- 10- The staff should have access to the exact numerical stock levels for every item in the database.
- 11- The system should support both automatic and manual updating of the shipping status of an item.
- 12- The system should generate statistical reports for the staff.

4.2 Non-Functional Requirements

- 1- The project must be developed using C# within the .NET framework.
- 2- The system must follow the Model-View-Controller (MVC) architectural pattern to separate data logic from the user interface.
- 3- The system must utilize SQLite as the primary relational database management system.
- 4- The system must consist of three distinct components:
 - Shared Backend: A core library for data logic and database access.
 - Web Application: For customer-facing features.
 - Desktop Application: For staff-facing features.
- 5- The system shall perform an automated full backup of the SQLite database file daily at 12:00 AM.
- 6- The entire user interface for both Web and Desktop applications must be presented in English.
- 7- The system must ensure that stock levels are decremented immediately upon successful order placement to prevent over-selling.

5. Database Schema



6. SQL Scripts

The project does not use hand-written SQL files. Instead, it uses Entity Framework Core as an ORM (Object-Relational Mapper). EF Core automatically creates the SQLite database and all tables from the C# model classes. At startup, `Database.EnsureCreated()` runs the schema-creation scripts that EF Core generates from the entity definitions in **AppDbContext**.

EF Core also translates the C# queries into SQL at run time. For example, the item search uses `EF.Functions.Like`, which EF Core turns into a SQLite LIKE statement:

```
query = query.Where(i => EF.Functions.Like(i.Name, $"%{name}%"));  
// Generated SQL: SELECT * FROM Items WHERE Name LIKE '%...%'
```

Table relationships and unique constraints are defined in code (in `OnModelCreating`) and applied to the database as `CREATE TABLE` / `CREATE UNIQUE INDEX` statements:

```
modelBuilder.Entity<User>().HasIndex(u => u.Username).IsUnique();  
modelBuilder.Entity<User>().HasIndex(u => u.Email).IsUnique();
```

For database backups, the system uses SQLite's official backup API (`source.BackupDatabase(dest)`)

7. LINQ Queries

Locations of LINQ queries and their purposes.

Backend

ItemService:

GetAllAsync - Search, filter and sort items. Staff and customers can find products by name (partial match), category, and price order.

GetByIdAsync - Fetch a single item by ID.

CreateAsync - Create a new inventory item (staff only).

UpdateAsync - Modify item details (staff only).

DeleteAsync - Remove an item from the inventory (staff only).

OrderService:

GetAllAsync - Retrieve orders with role-based filtering. Staff see all orders; customers see only their own.

GetByIdAsync - Get a specific order with all its items and product details.

CreateAsync - Create a new order with atomic stock deduction. Prevents overselling by checking and reducing stock in a single transaction.

UpdateStatusAsync - Update order shipping status (staff only).

ReportsService:

GetSalesReportAsync - Returns the top 5 of all time best-selling items by quantity.

UserService:

GetProfileAsync - Retrieve user profile data while limiting exposed fields.

UpdateProfileAsync - Update user contact and personal information.

GetAddressesAsync - Fetch all shipping addresses for a user.

AddAddressAsync - Add a new shipping address for the user.

DeleteAddressAsync - Remove a shipping address.

BackupService:

CleanupOldBackups - Clean up backup files older than 7 days to manage storage.

Web Application

HomeController:

Index - Build the product catalog view with category dropdown for filtering.

OrdersController:

AddToCart - Check if the item is already in the cart before adding.

RemoveFromCart - Find and remove items.

PlaceOrder - Transform shopping cart items into API order format.

History - Display order history with most recent orders first.

Desktop Application

Form1:

BindItems - Bind fetched items to DataGridView.

BindOrders - Bind fetched orders to DataGridView.

SortGrid - Enable user-driven column sorting with toggle direction.

GenerateSalesReportAsync - Fetches the sales report from backend API and displays total revenue, total orders count, and top-selling items list in the UI reports tab.

Desktop Notification Services

NotificationService:

CheckForNewOrders - Detect and fire event for new orders received, triggering a notification popup.

CheckForLowStock - Detect items that cross the low stock threshold and trigger a stock alert.

8. Delegates and Events

Desktop Application

NotificationService:

OrderPlacedNotificationHandler (delegate) - Defines the callback signature for new order alert popups.

LowStockDetectedHandler (delegate) - Defines the callback signature for low stock warning alerts.

OrderPlaced (event) - Triggered when background polling detects new, unseen customer orders.

LowStockDetected (event) - Triggered when a product's stock levels fall below the warning threshold.

NotificationCoordinator:

OrdersUpdated (event) - Triggered using a generic Action delegate to notify UI grids that a new order list was fetched.

ItemsUpdated (event) - Triggered using a generic Action delegate to notify UI catalog lists that new item data was fetched.

RefreshFailed (event) - Propagates background API fetch errors to trigger user-facing error dialogs.

LoginForm / Form1:

btnLogin.Click / btnRegister.Click (event subscriptions) - Bind button click events to trigger the asynchronous authentication logic.

btnRefreshItems.Click / btnRefreshOrders.Click (event subscriptions) - Bind manual refresh clicks to trigger API polling updates.

itemsGrid.SelectionChanged (event subscription) - Binds grid row selection changes to automatically populate the edit input fields.

_notificationTimer.Tick (event subscription) - Binds a recurring WinForms Timer tick to execute the background polling loop.

9. MVC Structure

The backend follows the Model–View–Controller (MVC) architectural pattern using ASP.NET Core. Responsibilities are clearly separated into distinct layers to maintain clean code and scalability.

9.1 Model

The data layer of the application. Entity classes define the database structure, while ApplicationDbContext manages database access. Data Transfer Objects (DTOs) define the exact shape of the data sent to and from clients.

- **9.1.1 Entity Classes (Backend/Models):** These map directly to the database tables.
 - Item, Order, OrderItem, User, Address
- **9.1.2 DTO Classes (Backend/DTOs):** These handle the data payloads for API requests and responses.
 - LoginRequest, RegisterRequest, ItemRequest, CreateOrderRequest, OrderItemRequest, UpdateOrderStatusRequest, UpdateUserRequest, AddressRequest, SalesReportResponse, TopSellingItem

9.2 View

In this architecture, the presentation layer is handled differently depending on the application context (API vs. Web).

- **9.2.1 Backend API:** In a REST API, there is no traditional "view." Instead, the view is the **JSON response** returned to the client applications (desktop or web).

- **9.2.2 Web Application (WebApp/Views):** The web frontend acts as a presentation layer consuming the API, utilizing Razor Pages for its views:
 - Home/Index.cshtml — Product catalog
 - Account/ — Login.cshtml, Register.cshtml, Profile.cshtml
 - Orders/ — Cart.cshtml, Checkout.cshtml, History.cshtml, Track.cshtml
 - Shared/ — _Layout.cshtml, Error.cshtml

9.3 Controller

Controllers act as the entry point for each HTTP request. They receive requests, delegate tasks to the matching service, and return a result. To keep controllers clean, core business logic is extracted into a separate **Service Layer** (e.g., ItemService, OrderService, AuthService, ReportsService).

- **9.3.1 Backend API Controllers (Backend/Controllers):**
 - AuthController — Handles user registration and login
 - ItemsController — Manages CRUD operations (list, create, update, delete) for items
 - OrdersController — Handles order creation and order status updates
 - UsersController — Manages user profiles and addresses
 - ReportsController — Generates sales reports
 - BackupController — Triggers manual database backups
- **9.3.2 Web Application Controllers (WebApp/Controllers):**
 - HomeController — Displays the product catalog
 - AccountController — Manages login, registration, and user profiles
 - OrdersController — Handles the shopping cart, checkout process, order history, and tracking

10. Sources

GitHub Repository: <https://github.com/serhatuzunbayir/InventoryOrderSPRING2025-26>

Instructions:

- 1- Clone the repository.
- 2- Restore dependencies: dotnet restore
- 3- Run Backend API: cd Backend
dotnet build
dotnet run - -project Backend.csproj
- 4- Run Web Application on a new terminal: cd WebApp
dotnet run
- 5- Run Desktop Application on a new terminal: cd DesktopApp
dotnet run

Version 1 video link: <https://youtu.be/k7gEKmvEww8>

Verson 2 video link: https://youtu.be/E_tWd28lOXA

11. User Interface Design

Inventory Staff Console

Inventory

Orders

Reports

Options

Search items

Clear

Id	Name	Quantity	Price (\$)	Category
1	Milk	119	\$10,00	Food
2	Cheese	99	\$12,00	Food

Item Details

Name

Category

Quantity

Price (\$)

Refresh

Add

Update

Delete

Staff: erkan API: http://localhost:5000

[Image 1 - Desktop Inventory]

Inventory Staff Console

Inventory

Orders

Reports

Options

Id	UserId	OrderedDate	Status
----	--------	-------------	--------

Refresh

Status

Pending

Update Status

Order Items

Itemid	ItemName	Quantity	Price
--------	----------	----------	-------

Staff: erkan API: http://localhost:5000

[Image 2 - Desktop Orders]

Inventory Staff Console

Inventory

Orders

Reports

Options

Total Revenue:

Total Orders:

Top Selling Items:

Total Sales Recorded

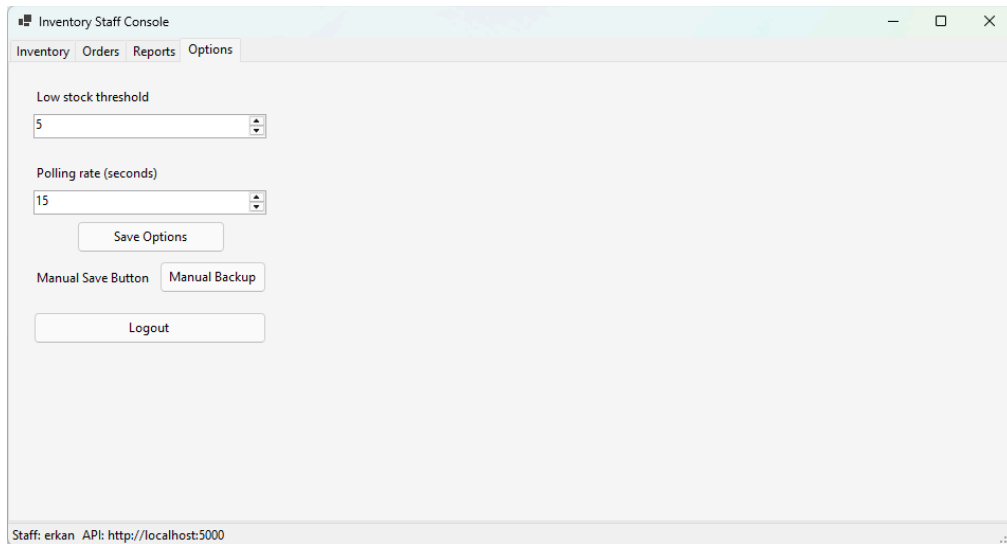
Generate Weekly Sale Trends

Select Item

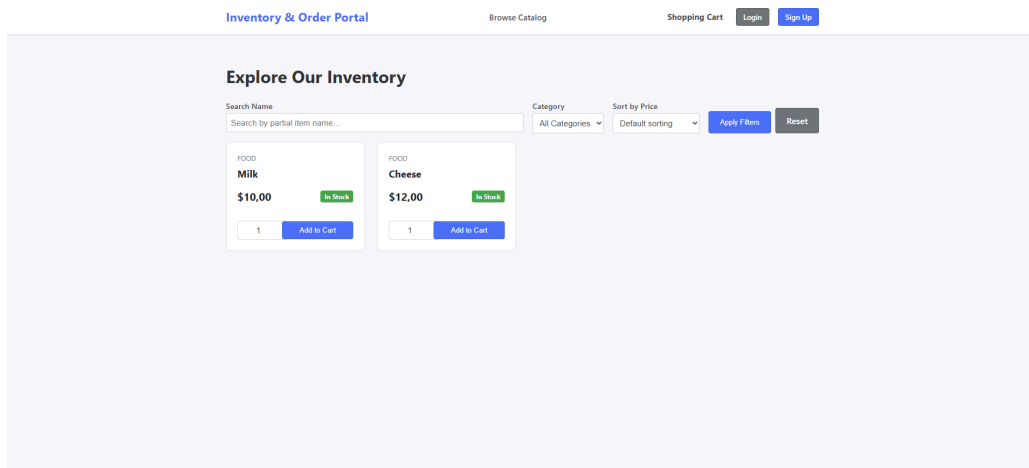
Weekly trend will appear here.

Staff: erkan API: http://localhost:5000

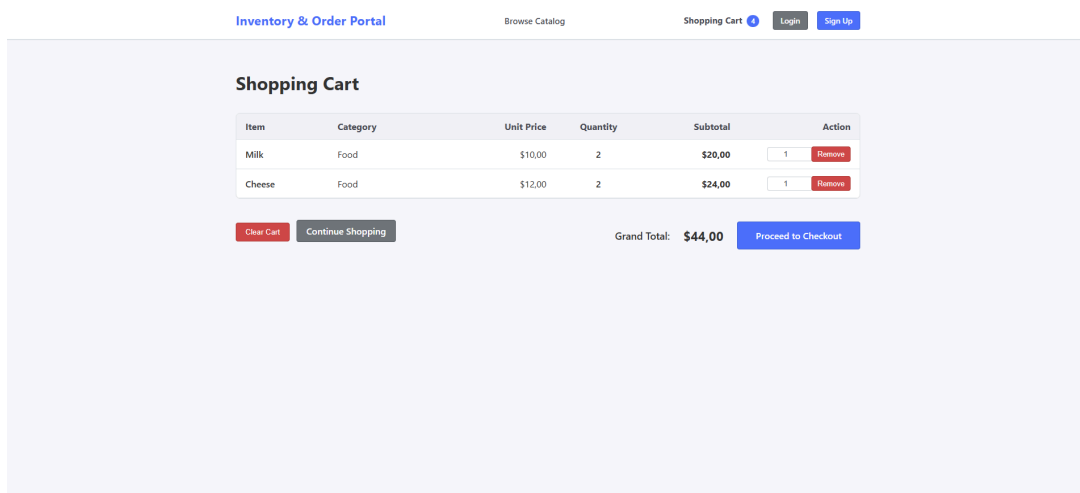
[Image 3 - Desktop Reports]



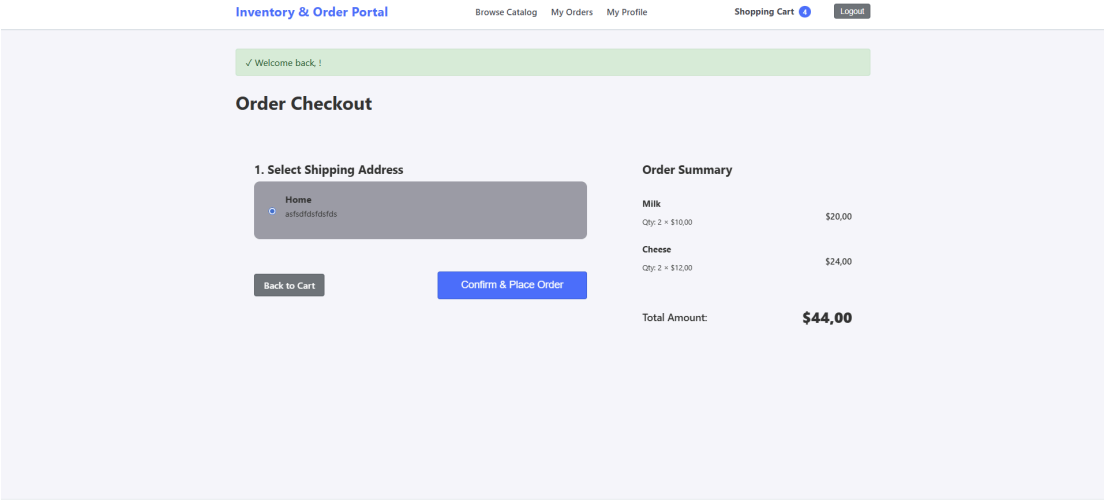
[Image 4 - Desktop Options]



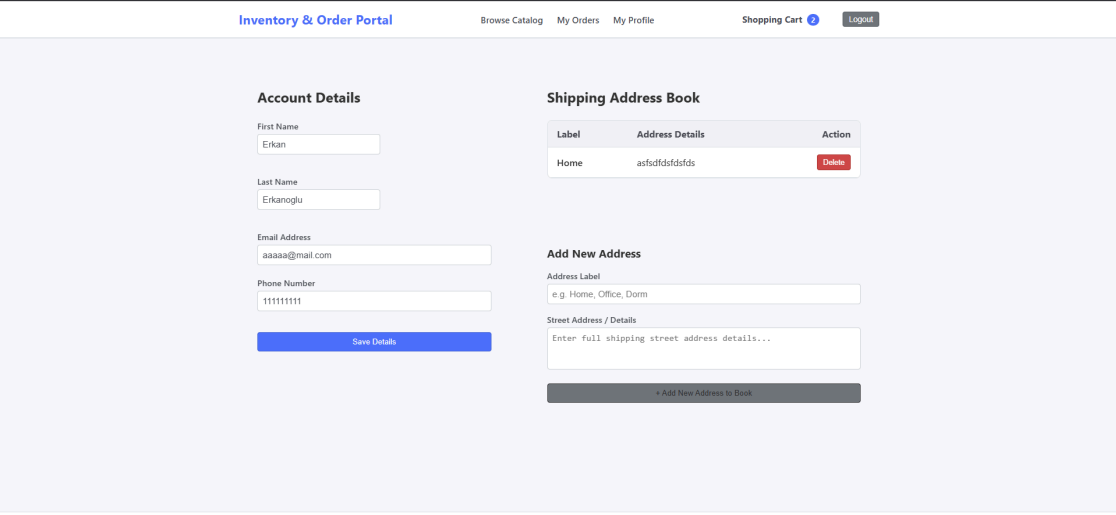
[Image 5 - Web Catalog]



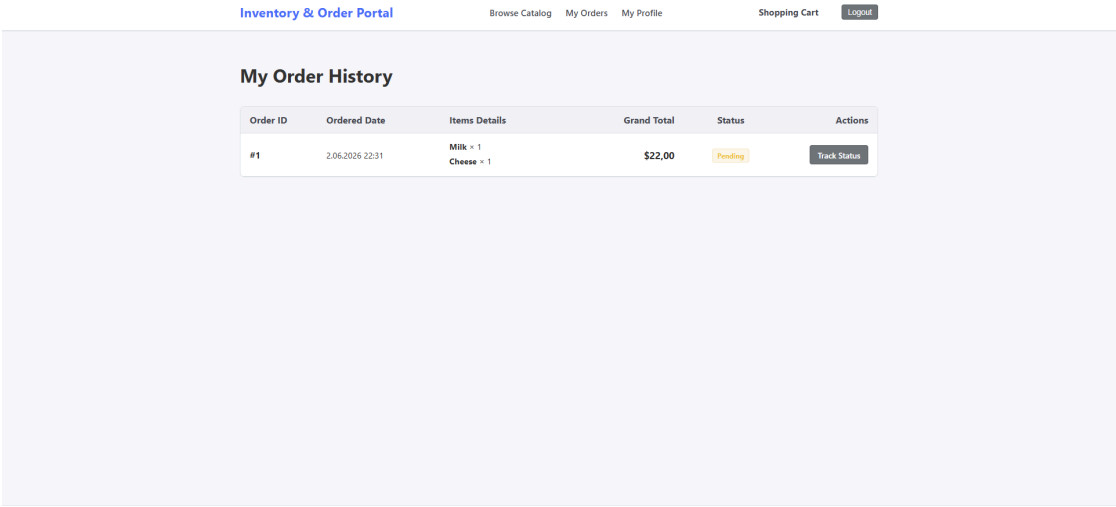
[Image 6 - Web Cart]



[Image 7 - Web Checkout]



[Image 8 - Web Profile]



[Image 9 - Web History]

[Image 10 - Web Login]

[Image 11 - Web Register]

12. Tests

Key tested scenarios (examples):

1. **Registration – new user:** Registration succeeds with valid information.
2. **Registration – duplicate username/email:** A second registration with the same username or email is rejected.
3. **Login – correct/wrong password:** Correct credentials return a token; a wrong password returns an error.
4. **Item filtering & sorting:** Filtering by name/category and ascending sort by price work correctly.
5. **Item CRUD:** Updating/deleting a non-existing ID returns null/false.
6. **Order – stock deduction:** A valid order correctly decreases the stock quantity (10→7).
7. **Order – insufficient stock / non-existing item:** Rejected with the appropriate error message.

8. **Order status:** A valid status is updated; an invalid status ("Flying") is rejected.
9. **Role-based access:** A customer sees only their own orders, while staff see all orders.
10. **Backup:** A non-empty backup file is created, data is copied, backups older than the retention window are deleted, and recent ones are kept.

Run command: dotnet test Backend.Tests/Backend.Tests.csproj

Result: Passed! - Failed: 0, Passed: 39, Skipped: 0, Total: 39

```
PS C:\Users\Hasan\Desktop\Git\InventoryOrder\Backend.Tests> dotnet test --logger "console;verbosity=detailed"
Passed Backend.Tests.OrderServiceTests.GetAll_StaffRole_ReturnsAllOrders [12 ms]
Passed Backend.Tests.BackupServiceTests.TakeBackup_KeepsRecentBackups [47 ms]
Passed Backend.Tests.OrderServiceTests.UpdateStatus_NonExistingOrder_ReturnsFail [20 ms]
Passed Backend.Tests.OrderServiceTests.Create_NonExistingItem_ReturnsFail [3 ms]
Passed Backend.Tests.AuthServiceTests.Register_DuplicateUsername_ReturnsFail [867 ms]
Passed Backend.Tests.AuthServiceTests.Register_DuplicateEmail_ReturnsFail [117 ms]
Passed Backend.Tests.AuthServiceTests.Register_NewUser_ReturnsSuccess [109 ms]
Passed Backend.Tests.AuthServiceTests.Login_ValidCredentials_ReturnsToken [237 ms]
[xUnit.net 00:00:01.66] Finished: Backend.Tests
Passed Backend.Tests.AuthServiceTests.Login_WrongPassword_ReturnsFail [212 ms]
Passed Backend.Tests.AuthServiceTests.Login_UnknownUser_ReturnsFail [4 ms]

Test Run Successful.
Total tests: 39
    Passed: 39
Total time: 2.0193 Seconds
```

13. Conclusion

In conclusion, the Inventory and Order Management System successfully provides a complete solution for small businesses to run more efficiently. By developing two separate interfaces, the system meets the needs of both staff and customers. The desktop application allows staff members to easily manage stock levels, process orders, and generate useful business reports. At the same time, the web application gives customers a simple way to view product availability, place new orders, and track their shipments.

Technically, the project safely manages data and uses important C# features. We successfully used LINQ to query the inventory database and implemented delegates to handle order processing smoothly. With key features like real-time inventory synchronization to prevent over-selling, the system passed all software tests. Overall, this project reached its main goals and delivered a reliable, working management tool.